学术论文写作 · 工程化排版

LaTeX Workshop 编译配置教程

从工具与配方，到完整论文工程

以官方 Compile 文档为主线
原理讲解 · 参数分类 · 完整实例 · 排错实验

学习目标
读懂 settings.json 中的调用关系；
配置引擎、参考文献、多文件工程与输出目录；
利用日志验证配置，并建立可复用的论文工作流。

中文学习版 / 2026 年 10 月 23 日
附可编译 LaTeX 源码与示例工程

目录

阅读指南：从能编译到理解编译 1

第 1 章：一次论文构建究竟发生了什么 2

1.1 把编辑器、插件和排版系统分开 . . . 2
1.2 为什么会生成这么多文件 . . . 2
1.3 先验证程序，再验证插件 . . . 3
1.4 从一条命令建立可验证的起点 . . . 3

第 2 章：配置写在哪里，怎样避免覆盖错误 4

2.1 用户设置与项目设置 . . . 4
2.2 先掌握 JSON 的结构 . . . 4
2.3 数组不会自动追加你的条目 . . . 4

第 3 章：tools 和 recipes 是配置的核心 5

3.1 tool 定义一个程序调用 . . . 5
3.2 recipe 定义有顺序的步骤 . . . 5
3.3 默认构建与手动选择 . . . 6
3.4 三个常见反例 . . . 6

第 4 章：读懂引擎选项与 latexmk 7

4.1 引擎选择首先服从论文模板 . . . 7
4.2 常用参数应当按程序分类 . . . 7
4.3 推荐起点：一个完整的 XeLaTeX 配置 . . . 8
4.4 把构建策略移入 latexmkrc . . . 8

第 5 章：占位符、工作目录与输出路径 9

5.1 用具体工程理解占位符 . . . 9
5.2 三种目录不能混为一谈 . . . 9
5.3 outDir 是双方约定 . . . 10
5.4 修改输出文件名时要两边一致 . . . 10

第 6 章：参考文献与多文件论文 11

6.1 为什么引用会出现问号 . . . 11
6.2 手动链条适合学习数据流 . . . 11
6.3 大论文如何确定主文件 . . . 12
6.4 搜索范围、依赖与 subfiles . . . 12

第 7 章：自动构建、文件监视与性能 13

7.1 自动构建何时触发 . . . 13
7.2 interval 与 delay 不是同一种计时器 . . . 13
7.3 两类忽略列表分工不同 . . . 13
7.4 避免同时使用两个文件监视器 . . . 14
7.5 把构建耗时拆成不同问题 . . . 14

第 8 章：读日志并建立排错顺序 15

8.1 区分三种信息 . . . 15
8.2 从第一条有意义的错误开始 . . . 15
8.3 警告与错误的处理优先级 . . . 15
8.4 一份可复用的诊断记录 . . . 16
8.5 一个输出目录失配的排错实例 . . . 16

第 9 章：辅助文件清理与可靠重建 17

9.1 清理是独立操作 . . . 17
9.2 latexmk 的小写 c 与大写 C . . . 17
9.3 自动清理与失败重试 . . . 17
9.4 修改清理参数为什么没有效果 . . . 18

9.5 把清理重建变成一次有目的的实验 18
第 10 章：魔法注释与外部构建 19
10.1 魔法注释是文件内的构建提示 19
10.2 TEX program 不等于完整 recipe 19
10.3 版本差异应当怎样处理 19
10.4 什么时候使用 external build 20
10.5 环境变量、Windows 与 WSL 20
第 11 章：可计算论文与其他进阶入口 21
11.1 R、Julia 和 Python 的共同模式 21
11.2 一个 Julia 流程的阅读示例 21
11.3 代码高亮与 shell escape 21
11.4 非常用设置怎样查 22
11.5 可计算论文还需要记录什么 22
第 12 章：综合实践与独立配置能力 23
12.1 实践目标 23
12.2 实践步骤 23
12.3 自测题 23
12.4 参考答案 23
12.5 形成自己的论文配置 24
附录 A：设置参数分类速查 25
附录 B：官方网页学习索引与版本说明 28
B.1 本书章节与官方入口 28
B.2 已发现的文档与定义差异 28
B.3 来源列表 28

阅读指南：从能编译到理解编译

这份教程面向使用 VS Code 和 LaTeX Workshop 撰写论文的读者。目标是让你能解释配置中每一个常用字段，选择适合论文模板的编译链，并从日志判断问题发生在哪一层。本文围绕官方 Compile 页的主题重新组织教学顺序，增加自编实例、反例、实验和检查方法；不是逐句翻译。

学习时准备一个独立的小项目，不要直接在临近提交的论文上试验。先学第 1 至 4 章，建立从命令到配置的联系；再学路径、参考文献与多文件工程；最后处理自动构建、诊断、清理和外部工具。附录给出参数速查与官方页面索引，适合日后查阅。

资料边界。 核对日期为 2026 年 10 月 23 日。主资料为官方 Compile Wiki；设置名、类型和默认值另与仓库 master 的 `package.json` 交叉检查，该快照声明版本为 `10.20.0`。master 不等于你已安装的稳定版。本文区分“官方默认值”和“教学建议值”，遇到差异时请以本机扩展的设置提示和实际日志为准。[S1][S2]

配套材料。 源码包含本教程的 LaTeX 源文件、编译说明、示例配置与一个可运行的英文小型论文工程。为使教程自身易于重编译，包内附中文字体；示例工程则用常见的 pdfLaTeX 与 BibTeX，便于在不同系统验证链路。

本书使用的术语。 “引擎”指 pdfTeX、XeTeX、LuaTeX 等排版程序；实际命令常是加载 LaTeX 格式的 `pdflatex`、`xelatex`、`lualatex`。“构建”可以包含多次排版、参考文献和索引处理；“编译”在日常交流中也常泛指整个构建过程。

第 1 章：一次论文构建究竟发生了什么

1.1 把编辑器、插件和排版系统分开

VS Code 编辑文件，LaTeX Workshop 组织构建、分析日志并连接预览器；TeX Live 或 MiKTeX 提供真正执行排版的程序和宏包。因此，装了插件不代表系统已经安装 `xelatex` 或 `latexmk`。遇到“程序找不到”时，应先检查程序安装和 PATH，而不是修改论文公式。

以一篇包含编号、目录和参考文献的论文为例：第一次排版读取主文件，写出中间记录；参考文献程序根据这些记录生成文献表；后续排版再读回结果，更新引用、目录和页码。页面布局变化可能继续影响交叉引用，所以一个构建过程可能执行同一排版命令多次。



图 1: 构建既包含顺序执行，也包含辅助数据的读回与更新。

1.2 为什么会生成这么多文件

文件	用途	实际意义
`.tex`、`.bib`	正文与参考文献数据库	属于论文源材料，应长期保存
`.aux`	标签、引用等辅助记录	多次排版之间的信息通道
`.toc`、`.lof`、`.lot`	目录、图目录、表目录数据	源文件更新后通常需要重新生成
`.bbl`	供排版使用的文献结果	可由数据库生成，也可能是投稿要求提交的材料
`.blg`、`.log`	文献程序与排版程序日志	报错时的重要证据
`.fls`	本次读取和写出的文件列表	有助于追踪依赖关系
`.fdb_latexmk`	latexmk 维护的构建信息	帮助判断哪些步骤需要重跑
`.synctex.gz`	源码位置与 PDF 位置的对应信息	正反向跳转所需
`.pdf`	排版结果	预览、检查与投稿使用

“删除中间文件后恢复正常”说明旧状态可能影响了构建，不意味着每次都应该清空中间文件。频繁删除会丢失增量构建信息，并让参考文献和交叉引用重新计算。

1.3 先验证程序，再验证插件

在 VS Code 的终端运行以下命令。只需检查实际打算使用的引擎；使用 BibTeX 或 Biber 的模板，再检查相应程序。

SH

```
latexmk -v
xelatex --version
bibtex --version
biber --version
```

能看到版本信息，说明当前终端可以找到程序；这还不能保证扩展宿主环境完全相同。特别是更新 PATH、使用 WSL、SSH 或容器时，要检查 LaTeX Workshop 所在环境。Windows 本机安装的 TeX 不会自动等同于 WSL 内安装的 TeX。[S1][S2]

实验 1。 先在终端直接编译一篇最小文档，再用插件构建同一个文件。如果终端成功而插件失败，优先比较程序路径、工作目录、参数和环境变量。这个实验能把论文错误与编辑器配置错误分开。

1.4 从一条命令建立可验证的起点

不要在尚未运行过最小文档时，同时添加文献链、自动清理、独立辅助目录和外部脚本。每加一层，就多出一个可能失配的接口。更清楚的学习顺序是：单文件成功，加入一个交叉引用，再加入文献，最后拆分章节与调整目录。

这里的“成功”也要具体化。程序正常结束只说明没有报告终止性错误；PDF 是否包含最新正文、公式是否越界、引用是否仍有问号，还需检查结果。构建命令和输出文件的时间应能对应起来，避免误把上一次留下的 PDF 当成本次成功的证据。

第 2 章：配置写在哪里，怎样避免覆盖错误

2.1 用户设置与项目设置

用户设置适用于自己的多个项目，适合字体大小、编辑习惯等个人偏好；项目设置适用于某篇论文，适合引擎、构建流程和输出目录。通常在论文根目录创建 `.vscode/settings.json`，让同一项目在重新打开后保持相同的编译配置。相同设置的项目值通常会覆盖用户值。[S4]

通过命令面板打开 `Preferences: Open Workspace Settings (JSON)` 可以编辑项目设置。通过 `Preferences: Open User Settings (JSON)` 可以编辑用户设置。设置页面搜索 `@ext:James-Yu.latex-workshop`，能够集中查看本机版本提供的选项、说明和合法取值。

2.2 先掌握 JSON 的结构

对象用花括号，数组用方括号，键与字符串值使用双引号。VS Code 设置文件支持 JSON with Comments，能写注释；如果把配置交给严格 JSON 解析器，则需删除注释及不允许的尾逗号。本文代码框标注 JSONC 时，表示 VS Code 设置片段。

JSONC

```
{
  "latex-workshop.latex.autoBuild.run": "onSave",
  "latex-workshop.latex.outDir": "build"
}
```

这段配置表示保存触发构建，并告知插件去 `build` 查找输出。它没有定义引擎，也没有确保某个自定义工具真的把文件写入 `build`。学习配置时，应逐项问：这一项影响插件行为，还是影响外部程序行为？

2.3 数组不会自动追加你的条目

对 `latex.tools`、`latex.recipes`、忽略列表等数组设置，自定义值通常是整个数组的替代值。不要假设写一个新工具后，所有内置工具仍会自动合并进来。若 recipe 引用了 `bibtex`，就应在生效的 tools 列表中保留其定义。

合并配置时，把新字段加入已有的最外层对象，不要在一个文件中连续粘贴两个独立对象，也不要重复定义同名键。有效的 JSON 并不等于有效的编译链：拼写正确、工具存在、参数合法、路径一致，缺一不可。

检查方法。 用鼠标悬停设置名，检查扩展说明；检查字符串是否属于可选取值；在“输出”面板确认本次真正使用的主文件和命令。不要只凭设置文件的外观判断生效情况。

第 3 章：tools 和 recipes 是配置的核心

3.1 tool 定义一个程序调用

tool 的基本字段可以这样阅读。[S2]

字段	要回答的问题	示例与注意事项
`name`	这个工具在配置中叫什么？	`xe-single` 是自定义标识，不要求等于程序名
`command`	启动哪个可执行程序？	`xelatex`，或该程序的完整路径
`args`	给程序传什么参数？	每个参数通常独立占一个数组元素
`env`	本次调用增加或覆盖什么环境变量？	如设置 TeX 搜索路径，通常可省略
`cwd`	程序从哪个目录执行？	可覆盖默认工作目录，通常可省略

JSONC

```jsonc
{
  "name": "xe-single",
  "command": "xelatex",
  "args": [
    "-synctex=1",
    "-interaction=nonstopmode",
    "-file-line-error",
    "%DOC_EXT%"
  ]
}
```

这只是一个 tool 对象，必须放进 `latex-workshop.latex.tools` 数组才是完整设置。名称为 `xe-single` 的工具运行一次 XeLaTeX；它的名称并不会自动赋予“完整处理参考文献”的能力。

3.2 recipe 定义有顺序的步骤

recipe 的 `name` 是显示给使用者的流程名称，`tools` 是按执行顺序排列的工具名。以下片段会依次运行一次 XeLaTeX、一次 BibTeX，再运行两次 XeLaTeX；前提是这两个工具名都已定义。

JSONC

```jsonc
{
  "name": "xe-bibtex-full",
  "tools": ["xe-single", "bibtex-main", "xe-single", "xe-single"]
}
```

这里重复工具名的意义是重复执行，不是重复声明。最后两次排版用于读入文献结果并稳定引用；固定执行次数只是常见流程，不保证适用于所有索引、目录或复杂依赖。若需求是“自动判断需要哪些步骤”，优先考虑 latexmk。[S3]

3.3 默认构建与手动选择

`latex-workshop.latex.recipe.default` 有三个常见用法：`first` 选择列表第一项；`lastUsed` 选择最近使用的流程；具体 recipe 名把默认值固定到那个流程。普通 Build 与自动构建会参考该设置，但外部构建命令和魔法注释可能改变选择路径，详见第 10 章。

使用命令面板的 `LaTeX Workshop: Build with recipe`，可以明确选择一次构建使用哪个流程。Windows/Linux 常见构建快捷键是 Ctrl+Alt+B；快捷键可被用户修改，以命令面板为准。停止构建时使用插件的终止编译命令，避免在卡住的流程上不断追加构建请求。[S1]

3.4 三个常见反例

反例一：名称引用不匹配。 定义了 `xe-single`，recipe 却写成 `xelatex`。应修改引用或定义对应名称，不要把显示名与可执行文件名混淆。

反例二：把整条命令放进 command。 `"command": "xelatex -synctex=1 main.tex"` 不是规范的普通工具定义；应把程序与参数分开。

反例三：把多个普通参数合并成一个字符串。 `"args": ["-synctex=1 -file-line-error", "%DOC_EXT%"]` 不等同于两个独立选项。普通 tool 的参数数组应按调用边界拆分；魔法 options 和显式 shell 脚本属于不同调用情境。

练习 3。 新增一个名为 `xe-two-pass` 的 recipe，让它连续运行两次 `xe-single`。只需要修改 recipes，不需要复制工具定义。这样可验证“工具复用”与“流程组合”的区别。

第 4 章：读懂引擎选项与 latexmk

4.1 引擎选择首先服从论文模板

英文期刊模板可能要求 pdfLaTeX；含中文和系统字体的模板常使用 XeLaTeX 或 LuaLaTeX。模板文档、宏包要求、字体方案比“哪种引擎更新”更能决定选择。切换引擎后应重新检查字体、换行和页面结果，不应假设不同引擎生成的 PDF 完全一致。

latexmk 是构建调度工具，不是另一个与 XeLaTeX 并列的排版引擎。`latexmk -xelatex` 表示让 latexmk 组织构建并调用 XeLaTeX。它可以按需重跑引擎、处理 BibTeX/Biber 等依赖；具体行为也受 `.latexmkrc` 和项目结构影响。[S3]

4.2 常用参数应当按程序分类

参数	所属情境	作用与误区
`-pdf`	latexmk	选择通常由 pdfLaTeX 生成 PDF 的构建模式；不应随意与其他引擎模式叠加
`-xelatex`	latexmk	选择 XeLaTeX 流程并生成 PDF
`-lualatex`	latexmk	选择 LuaLaTeX 流程并生成 PDF
`-synctex=1`	支持该选项的引擎；latexmk 可转交	生成压缩同步定位信息
`-interaction=nonstopmode`	引擎；latexmk 可转交	避免遇错停在交互提示符，仍会报告错误
`-file-line-error`	引擎；latexmk 可转交	在错误消息中提供文件和行号
`-halt-on-error`	引擎；latexmk 可转交	首次错误时尽早停止，适合快速暴露根因
`-outdir=build`	latexmk	指定构建输出目录
`-output-directory=build`	常见 TeX 引擎	直接调用引擎时指定输出目录
`-jobname=paper`	引擎及相关构建工具	修改生成文件的基名，需让插件同步知道
`-shell-escape`	支持的引擎/工具链	允许文档调用外部命令，只在实际需要时使用

具体选项的合法拼写仍应查询对应程序的 `--help` 或手册。一个参数在 latexmk 中可用，不等于复制到 BibTeX、Biber 或另一种工具中仍然可用。`nonstopmode` 解决“不要停下来等输入”，`halt-on-error` 解决“发生错误就结束本次调用”，二者并不矛盾。[S3]

4.3 推荐起点：一个完整的 XeLaTeX 配置

以下为本书建议的入门设置，不是官方默认配置的逐字副本。它适用于模板允许 XeLaTeX、已安装 latexmk 的项目。程序依赖和模板宏包仍须安装。

JSONC

```
{
  "latex-workshop.latex.tools": [
    {
      "name": "mk-xe",
      "command": "latexmk",
      "args": [
        "-xelatex",
        "-synctex=1",
        "-interaction=nonstopmode",
        "-file-line-error",
        "-outdir=%OUTDIR%",
        "%DOC_EXT%"
      ]
    }
  ],
  "latex-workshop.latex.recipes": [
    {"name": "paper-xe", "tools": ["mk-xe"]}
  ],
  "latex-workshop.latex.recipe.default": "paper-xe",
  "latex-workshop.latex.outDir": "%DIR%/build",
  "latex-workshop.latex.autoBuild.run": "onSave",
  "latex-workshop.latex.autoClean.run": "never"
}
```

读这段配置时，从 recipe 的 `tools` 找到 `mk-xe`，再找到 `command` 与参数。输出目录明确锚定到主文件目录。其余高级选项暂时保留默认值，让第一次排错涉及的变量尽可能少。

4.4 把构建策略移入 latexmkrc

当终端、编辑器与持续集成都需要同样的构建规则时，可以将通用规则放进项目根目录的 `.latexmkrc`。例如，以下 Perl 赋值选择 XeLaTeX 模式并打开 SyncTeX。它不是 JSON，也不是 LaTeX 代码。

PERL

```
$pdf_mode = 5;
$synctex = 1;
```

相应工具可只传主文件等通用参数；也可沿用显式 `-xelatex` 配置，但应确保两个位置没有互相冲突的引擎或目录设置。latexmk 会读取一定位置的初始化文件，实际加载顺序和变量含义见手册的初始化文件章节。不要同时在多个位置维护两套不一致的构建策略。[S3]

第 5 章：占位符、工作目录与输出路径

5.1 用具体工程理解占位符

假设工作区是 `/home/me/thesis`，主文件是 `/home/me/thesis/main.tex`，输出目录设置为 `%DIR%/build`。下表展示占位符在普通本机工程中的典型展开。[S1][S2]

占位符	对应含义	本例的展开
`%DOC%`	主文件完整路径，不含扩展名	`/home/me/thesis/main`
`%DOC_EXT%`	主文件完整路径，含扩展名	`/home/me/thesis/main.tex`
`%DOCFILE%`	主文件名，不含目录和扩展名	`main`
`%DOCFILE_EXT%`	主文件名，含扩展名	`main.tex`
`%DIR%`	主文件所在目录	`/home/me/thesis`
`%OUTDIR%`	配置的输出目录	`/home/me/thesis/build`
`%AUXDIR%`	辅助文件目录；默认引用 OUTDIR	本例通常与输出目录相同
`%WORKSPACE_FOLDER%`	工作区路径	`/home/me/thesis`
`%TMPDIR%`	插件使用的临时目录	由运行环境决定

`%DOC%` 省略扩展名通常仍能被 TeX 程序接受，因此官方示例中很常见；如果调用的程序要求完整文件名，使用 `%DOC_EXT%` 更直接。BibTeX 常接收不带 `.aux` 后缀的基名；Biber 也通常接收对应作业基名。不要因为参数外观相似就随意互换。

此外还有 Windows 分隔符版本 `%DOC_W32%`、`%DOC_EXT_W32%`、`%DIR_W32%`、`%OUTDIR_W32%`，以及相对路径 `%RELATIVE_DIR%`、`%RELATIVE_DOC%`、`%RELATIVE_CWD_DIR%`、`%RELATIVE_CWD_DOC%`。相对路径变体是否带扩展名，应按官方定义核对；这些 DOC 变体通常不带 `.tex`。多根工作区与 Docker 还可能改变上下文，优先从日志验证实际展开。[S1][S5]

5.2 三种目录不能混为一谈

主文件目录决定项目入口的位置。**工作目录 cwd**决定外部程序怎样解释相对输入路径。**输出目录**决定生成文件落在哪。把 cwd 改成 `build` 不等于正确设置输出目录，也可能让 `figures/a.pdf` 和 `refs.bib` 不再按预期找到。

`latex-workshop.latex.build.fromFolder` 指定整个 recipe 的执行位置；空字符串通常表示主文件目录。相对值按包含主文件的工作区目录解析，没有工作区时按主文件目录解析。tool 的 `cwd` 可以对单步再做覆盖。[S2][S5]

JSONC

```jsonc
{
  "latex-workshop.latex.build.fromFolder": "",
  "latex-workshop.latex.outDir": "%DIR%/build",
  "latex-workshop.latex.auxDir": "%OUTDIR%"
}
```

上面的设置片段保持主文件目录为默认工作目录，并将输出和辅助文件放在一起。开始学习时不必把 `.aux` 与 PDF 强行分离；分离后引擎、文献程序、索引程序和插件都需要对目录形成一致理解。

5.3 outDir 是双方约定

插件需要知道“去哪里找 PDF”，外部程序需要知道“把 PDF 写到哪里”。所以自定义 latexmk 工具时，常将 `latex.outDir` 与 `-outdir=%OUTDIR%` 配合。仅修改插件设置、却让工具继续输出到别处，会出现“编译似乎成功，预览却找不到 PDF”的现象。

直接调用 XeLaTeX 时通常使用 `-output-directory=%OUTDIR%`，并提前创建目标目录；不要假设所有引擎都会创建目录。使用 `.latexmkrc` 设置目录时，也要显式核对插件的 `outDir`。不能假设插件必然解析 rc 文件得知你的目录布局。[S2][S3]

5.4 修改输出文件名时要两边一致

若工具使用 `-jobname=paper-final`，生成的文件基名会改变。还应设置 `latex-workshop.latex.jobname` 为 `paper-final`，让插件正确定位 PDF 与同步文件。这个配置不应写成带占位符的表达式，也不会替你向程序自动添加所有必要参数。[S2]

实验 5。 把输出放进 `build`，确认 PDF、日志与同步文件的位置。然后只改插件的 outDir，不改工具参数，观察日志和预览；最后恢复一致。实验目的在于理解两个设置各自承担的职责。

第 6 章：参考文献与多文件论文

6.1 为什么引用会出现问号

文献数据库只是原始记录，排版系统还需要知道本次引用了哪些记录、用什么样式排列它们，并将结果重新纳入正文。引用出现问号可能是构建步骤不完整，也可能是键名拼错、数据库路径错误、文献程序失败或样式不兼容。先读相关日志，不要机械重复点击编译。

常见传统流程使用 `\bibliographystyle{...}` 和 `\bibliography{...}`，通常配合 BibTeX；使用 biblatex 的工程应根据 `backend` 选项选择 Biber 或 BibTeX。Biber 与 BibTeX 不是仅通过名称就能互换的两个程序；应先读论文模板的文献配置。

6.2 手动链条适合学习数据流

下面是固定链条的最小构成。为避免引入额外目录问题，本例将中间文件留在主文件目录。把对象分别放入相应 tools 和 recipes 数组。

JSONC

```
// tools
{
  "name": "pdf-single",
  "command": "pdflatex",
  "args": ["-synctex=1", "-interaction=nonstopmode",
           "-file-line-error", "%DOC_EXT%"]
},
{
  "name": "bib-main",
  "command": "bibtex",
  "args": ["%DOCFILE%"]
}
// recipe
{
  "name": "pdf-bibtex-full",
  "tools": ["pdf-single", "bib-main", "pdf-single", "pdf-single"]
}
```

第一次 `pdf-single` 产生供 BibTeX 使用的辅助信息，`bib-main` 产生文献表，之后的排版读入文献表并更新引用。如果改用独立输出目录，BibTeX 需要找到该目录中的 `.aux`，同时仍能找到 `.bib` 和 `.bst`。此时优先交给 latexmk 协调，或认真设置参数、cwd 与搜索路径，不要只把 `%DOCFILE%` 换成任意路径。

6.3 大论文如何确定主文件

把摘要、相关工作、建模和实验分成章节文件，并由主文件通过 `\input` 或 `\include` 引入，可以减少编辑负担。主文件包含文档类、导言区和正文边界，章节文件通常只有章节正文。

LATEX

```
% main.tex
\documentclass{article}
\begin{document}
\input{chapters/introduction}
\input{chapters/model}
\end{document}
```

LATEX

```
% chapters/model.tex
% !TEX root = ../main.tex
\section{Model}
The model is described here.
```

根文件魔法注释的路径相对于当前章节文件；打开 `model.tex` 时，它仍可将构建指向上一级的 `main.tex`。`enableMagicComments` 必须允许这一机制。插件还会根据文档类和文件包含关系尝试自动找主文件，但明确入口通常更容易维护。[S1]

6.4 搜索范围、依赖与 subfiles

`latex.search.rootFiles.include` 与 `exclude` 控制主文件候选搜索。若工作区包含模板样例、旧稿和多篇论文，可缩小范围以避免选错入口。注意它们控制的是插件搜索，不会代替 TeX 的输入文件搜索。

`latex.texDirs` 是插件查找额外 TeX 输入的目录列表；`latex.bibDirs` 主要用于文献补全搜索。编译器本身是否能找到文件，还可能需要 TeX 的 `TEXINPUTS` 或 `BIBINPUTS` 环境变量。一个文件能在编辑器补全，不代表外部程序一定能读取它。[S2]

如果采用 subfiles 宏包，子文件也可能独立排版。`rootFile.useSubFile` 影响自动构建、清理和同步的目标，`rootFile.doNotPrompt` 影响交互时是否询问。普通的 input 章节不需要为了“分文件”就引入 subfiles。先确定是否真的需要单章独立输出，再配置相应行为。

检查方法。 在插件输出中记录实际 root file，再观察依赖是否包含正在编辑的章节。`.fls` 文件还能提供本次实际使用的依赖线索。若主文件识别正确但图片仍找不到，应继续检查 cwd 与 TeX 路径，不能停在主文件识别这一层。

第 7 章：自动构建、文件监视与性能

7.1 自动构建何时触发

`latex.autoBuild.run` 的官方默认值为 `onFileChange`。对初学者，本书建议先使用 `onSave`，以便将一次保存与一次构建联系起来；需要观察外部程序更新图片或文献时，再考虑 `onFileChange`。[S2]

取值	触发方式	使用情境
`never`	不自动构建	调试、集中写作、耗时较大的工程
`onSave`	在 VS Code 中保存相关 TeX 文件时触发	希望构建时机清楚可控
`onFileChange`	监视到项目依赖在磁盘上变化时触发	图片或数据库由其他程序更新

`onFileChange` 不等于“键盘每敲一个字就构建”。若启用了 VS Code 自动保存，编辑、保存与磁盘变化可能相继发生，使你感觉插件一直在运行。排查频繁构建时，同时检查编辑器自动保存与外部生成程序。

7.2 interval 与 delay 不是同一种计时器

`autoBuild.interval` 控制自动构建的最小触发间隔，默认 1000 ms；它不是“停止打字后等待多久”的同义词，也不能保证每次修改都会形成一个排队的完整构建。它限制频率，不会让引擎本身运行得更快。[S2][S6]

`watch.delay` 与确认非二进制文件删除有关，适合处理“先删后写”的保存模式；`watch.pdf.delay` 针对 PDF 等二进制变化的稳定确认，可能有助于减少频繁刷新。它们也不是通用的自动编译等待时间。

7.3 两类忽略列表分工不同

`autoBuild.onSave.files.ignore` 只影响保存触发的忽略规则，默认通常忽略 `.sty`、`.cls`；`watch.files.ignore` 控制文件监视路径的忽略。规则通常按绝对路径做 glob 匹配。`**/*.sty` 可匹配任意层级的样式文件，`**/build/**` 可匹配 build 目录下的内容。

JSONC

```
{
  "latex-workshop.latex.autoBuild.run": "onSave",
  "latex-workshop.latex.autoBuild.interval": 1500,
  "latex-workshop.latex.autoBuild.onSave.files.ignore": [
    "**/*.sty", "**/*.cls"
  ]
}
```

本例的 1500 是教学建议值，默认值仍是 1000。自定义完整忽略列表会替代原数组，尤其修改 `watch.files.ignore` 时，应先查看并保留仍需生效的默认规则。

7.4 避免同时使用两个文件监视器

latexmk 的 `-pvc` 会长期监视文件并重编译；LaTeX Workshop 自己也有自动构建机制。常规插件 recipe 中通常让 latexmk 完成一次构建后退出，由插件负责下一次触发；没有明确需求时，不必再加 `-pvc`。否则可能出现一直显示构建中、重复监视或难以终止的现象。[S3]

实验 7。 先设为 `never`，手动构建成功；改为 `onSave`，保存一次确认日志；再改为 `onFileChange`，由外部程序更新一张被引用的图片，观察是否触发。每次只改变一个条件，才能理解触发来源。

7.5 把构建耗时拆成不同问题

构建慢可能来自频繁触发，也可能来自每次确实要处理很多内容。前者看保存、监视与节流；后者看引擎调用次数、大图片处理、代码前处理及文献和索引步骤。增大 interval 只减少触发机会，不会缩短一次必要构建。

可以记录三类耗时：首次完整构建、只修改一行正文后的构建、没有修改输入的重复构建。它们回答的是不同问题。首次构建可能包含字体、缓存和所有文献步骤；无改动重建则主要检验依赖判断。比较不同配置时应使用同一文档和相同修改条件，不能把“清理后第一次”与“已有缓存的第二次”直接比较。

第 8 章：读日志并建立排错顺序

8.1 区分三种信息

插件日志说明主文件、recipe、程序调用和路径；编译器输出说明排版程序的实际运行；Problems 面板汇总插件解析出的错误与警告。Problems 便于跳转，但判断完整原因时仍应保留原始输出。可在“输出”面板切换 LaTeX Workshop 与 LaTeX Compiler，或使用查看编译日志命令。[S1]

调试多步 recipe 时，可将 `latex.build.clearLog.everyRecipeStep.enabled` 设为 `false`，保留每一步输出。否则后一步的日志可能覆盖你正在寻找的 BibTeX 或第一遍排版信息。[S2]

8.2 从第一条有意义的错误开始

例如一个未定义命令可能导致后续十几条“缺少括号”“缺少美元符号”。从最早出现、能指向具体源文件的错误开始修复，然后重新构建，通常比逐条消除后续消息有效。`-file-line-error` 有助于形成带文件名和行号的诊断。

现象	优先怀疑	下一步检查
`ENOENT` 或无法启动程序	command、安装位置、PATH	在同一环境检查程序版本与完整路径
缺少 `.sty`	宏包未安装或搜索路径不正确	查询发行版安装情况与 TeX 搜索路径
fontspec 提示需要 XeTeX/LuaTeX	使用了不兼容引擎	查看实际 command 与引擎模式
引用为问号	文献链条或数据库问题	检查 `.blg`、Biber 日志、键名与 backend
编译成功但无 PDF 预览	输出目录、jobname 或文件锁	找到实际 PDF，与插件预期路径比较
子章节无法编译	主文件选错或直接编了片段	检查 root file 与根文件注释
一直显示构建中	交互等待、长期监视或外部程序	检查最后一条输出与是否使用 `-pvc`
PDF 更新但无法跳转	缺少或不匹配的 SyncTeX	检查同步文件位置、时间与 PDF 是否对应

8.3 警告与错误的处理优先级

`Overfull` 表示内容超出排版盒子，论文中可能表现为长公式、长 URL 或表格越界；`Underfull` 通常反映伸缩不足或排版松散。二者不必然使构建失败，但提交前应结合 PDF 视觉检查。

`message.badbox.show` 可以选择显示 `both`、`overfull`、`underfull` 或 `none`。各种 `message.*log.exclude` 按正则过滤面板诊断；这些选项改变“显示什么”，不会修复论文中的问题。`message.error.show` 等开关主要控制通知，也不代表程序停止报错。[S2]

8.4 一份可复用的诊断记录

每次记录：主文件绝对路径、工作目录、recipe 名、实际 command 与 args、首条错误、PDF 预期位置和实际位置。若寻求别人帮助，附最小复现工程和这些信息，往往比粘贴全部用户设置更有效。

排错顺序。 先检查程序是否能启动；再核对入口文件与命令；随后处理首条编译错误；最后核对输出路径、预览和同步。只有证据指向旧辅助数据时，才执行清理并重建。

8.5 一个输出目录失配的排错实例

设论文主文件是 `main.tex`，插件期望 `build/main.pdf`，但自定义工具的参数只有主文件，没有输出目录选项。你点击 Build 后看见程序正常结束，预览仍显示旧版本。此时不应先怀疑 PDF 阅读器缓存；先在磁盘上比较 `main.pdf` 与 `build/main.pdf` 的修改时间。

如果最新结果在主文件旁，就说明引擎已成功工作，失配发生在“实际输出”和“插件查找”之间。给工具补上正确的输出参数，或将插件查找位置恢复为实际输出位置，再构建并比较时间。若还设置了 jobname，也要把文件基名一起纳入检查。

这个实例体现一种可迁移的方法：先确定已经正常工作的环节，再找到第一个不一致的接口。程序安装、论文内容、文件位置与预览同步各有证据，不能把所有现象都归为“插件坏了”。

第 9 章：辅助文件清理与可靠重建

9.1 清理是独立操作

插件可调用 `Clean up auxiliary files` 清理。`latex.clean.method` 决定采用外部命令还是 glob 文件匹配，官方默认是 `command`。当 method 为 command 时，应看 `clean.command` 和 `clean.args`；当 method 为 glob 时，才主要看 `clean.fileTypes`。[S2]

JSONC

```
{
  "latex-workshop.latex.clean.method": "command",
  "latex-workshop.latex.clean.command": "latexmk",
  "latex-workshop.latex.clean.args": [
    "-outdir=%OUTDIR%", "-auxdir=%AUXDIR%", "-c", "%DOC_EXT%"
  ],
  "latex-workshop.latex.autoClean.run": "never"
}
```

此处显式用主文件作为清理目标。官方默认 clean.args 还提供 `%TEX%`，表示调用清理时所处 TeX 文件的完整路径；它不同于 recipe 中表示项目主文件的 DOC 占位符。多文件工程清理时应确认实际目标。

9.2 latexmk 的小写 c 与大写 C

`latexmk -c` 清理可再生成的辅助结果，通常保留最终 PDF；`latexmk -C` 还会清理最终输出。究竟哪些文件属于可再生成文件，还受 latexmk 配置和数据库可用性影响。尤其 `.bbl` 可能是投稿时唯一随源文件提供的文献结果，应先判断是否有能力重建。[S3]

如果选择 glob 方法，建议从明确的辅助扩展名开始，例如 `%DOCFILE%.aux`、`%DOCFILE%.log`；不要将正文 `.tex`、数据库 `.bib` 或图片加入清理规则。`clean.subfolder.enabled` 影响子目录范围。目录模式与文件模式有区别，匹配空目录通常需要末尾斜杠，复杂规则应先在小型工程验证。

9.3 自动清理与失败重试

`autoClean.run` 负责何时自动清理；`autoBuild.cleanAndRetry.enabled` 负责构建出错后的清理重试，两者不是同一个选项。对排错阶段，保留辅助文件和日志更有帮助；若反复自动清理使关键证据消失，可临时关闭清理重试。[S2]

JSONC

```
{
  "latex-workshop.latex.autoClean.run": "never",
  "latex-workshop.latex.autoBuild.cleanAndRetry.enabled": false
}
```

本书建议不要为了文件夹“看起来整洁”而每次成功后清空所有数据。将生成文件放进 build 目录通常已经能改善管理，并保留增量构建收益。投稿前再按要求整理源文件与必要的生成结果。

9.4 修改清理参数为什么没有效果

典型情况是修改了 `clean.fileTypes`，但 method 仍为 command；或者修改了 latexmk 清理参数，实际却采用 glob；也可能仅修改用户设置，被项目设置覆盖。先找到生效的 method，再观察清理输出，确认目录与目标文件。

9.5 把清理重建变成一次有目的的实验

先保留失败日志，记下首条错误和使用的引擎。随后确认数据库、图片与模板均存在，再清理可重建的辅助文件并完整构建。如果问题消失，继续判断是否与切换引擎、修改文献后端或移动目录有关；不要把“清理有效”直接当成问题根因。

若清理后仍出现相同首条错误，说明继续删除同一批文件通常不会带来新信息，应回到源代码、程序参数或环境检查。若只剩引用问号，则检查文献步骤是否真正执行，以及数据库或样式是否能被找到。每一次操作都应回答一个问题，这样排错记录才能在下次复用。

第 10 章：魔法注释与外部构建

10.1 魔法注释是文件内的构建提示

它们看起来是注释，对 TeX 引擎通常没有排版作用，但编辑器可解释其含义。以下两个例子分别指定根文件和已有 recipe。recipe 名必须与设置中完全一致。[S1][S7]

LATEX

```
% !TEX root = ../main.tex
% !LW recipe = paper-xe
```

主文件选择和 recipe 选择解决不同问题。前者确定“编哪篇论文”，后者确定“怎样编”。不应为了修复主文件识别而不断更改引擎。

10.2 TEX program 不等于完整 recipe

LATEX

```
% !TEX program = xelatex
```

该提示在受支持且启用的调用路径中，可令插件直接选择程序。仅有程序注释时，不应推断会自动执行完整文献流程。兼容机制还支持 `% !BIB program`、TEX/BIB options，以及 `latex.magic.args`、`latex.magic.bib.args`。options 一旦覆盖默认参数，必须自行保留需要的输入文件等参数。

对日常论文，优先用清楚命名的 recipe；若要在文件中指定它，用 LW recipe 注释即可。不要同时叠加多种互相冲突的选择方式。显式“Build with recipe”通常优先于程序/配方选择注释，但不意味着文件入口的根注释也失去意义。

10.3 版本差异应当怎样处理

本次核对中，Wiki 与 master 的 `recipe.ts` 在同时存在 TEX program 和 LW recipe 时给出的选择顺序不完全一致；设置 schema 中还保留了 `build.forceRecipeUsage`，但单靠其声明不能推断当前调用路径必然忽略程序注释。因此本书不建议依赖冲突注释的隐含优先级，也不把某个历史开关当作通用修复方式。[S1][S2][S7]

可靠做法是：移除不需要的程序注释；明确选择 recipe；在日志中核对真正执行的程序。若确实依赖某个魔法注释特性，记录本机扩展版本，用最小项目验证。`build.enableMagicComments` 会影响包括根提示在内的相关机制，不能在未评估多文件入口前随意关闭。

10.4 什么时候使用 external build

当论文由 Makefile 或统一脚本负责生成图表、运行代码、排版和复制结果时，外部构建更容易复用既有流程。设置非空 `latex.external.build.command` 后，会绕过普通 recipes 与通常的根文件发现路径；调用位置一般是工作区目录。脚本应明确自己的入口，不能假设一定获得正常 recipe 的全部上下文。[S1][S2]

JSONC

```
{
  "latex-workshop.latex.external.build.command": "python3",
  "latex-workshop.latex.external.build.args": [
    "scripts/build_paper.py", "main.tex"
  ]
}
```

本例中的脚本必须由项目提供。命令参数中的 `%DOC_EXT%` 等占位符只有在相应主文件上下文已知时才有意义。若外部命令设置仍生效，修改 recipes 可能不会改变普通 Build 的行为。返回 recipe 机制时，将 external command 恢复为空字符串。

10.5 环境变量、Windows 与 WSL

`env` 为单次工具调用提供环境变量覆盖。TeX 的路径变量语法与平台有关，例如路径分隔符、递归搜索和保留默认搜索路径的写法都需要查 TeX 发行版文档；不要把插件的占位符替换机制当成 shell 通用的变量展开。

Windows 路径可在 JSON 中写为 `C:/texlive/.../xelatex.exe`，避免反斜杠转义混乱；若使用反斜杠，需要写成 `\\`。普通 args 中的含空格路径仍应作为一个元素传入，通常不再手工添加一层 shell 引号。WSL 调用 Windows 程序时还涉及 `.exe` 名称和路径体系，优先选择在同一环境内安装并运行整套工具链。

第 11 章：可计算论文与其他进阶入口

11.1 R、Julia 和 Python 的共同模式

官方 Compile 页还介绍 `.rnw` 与 `.jnw`：先运行代码织入工具，将带代码的文档转换成 `.tex`，再用常规构建工具生成 PDF。这仍然是“多个 tools 按顺序组成 recipe”的同一个机制，新增的是前处理步骤。[S1][S2]

输入与语言	典型工具	需要额外准备什么
R / `.Rnw`	Rscript 与 knitr	R 环境、knitr、数据与 R 代码依赖
Julia / `.jnw`	Julia 与 Weave.jl	Julia、Weave、数据与 Julia 包
Python / `.pnw`	Pweave	Python 环境、Pweave 与相应包
普通 `.tex`	latexmk 或直接引擎	TeX 发行版与模板依赖

11.2 一个 Julia 流程的阅读示例

JSONC

```
{
  "name": "weave-jnw",
  "command": "julia",
  "args": [
    "-e",
    "using Weave; weave(\"%DOC_EXT%\", doctype=\"tex\")"
  ]
}
```

`-e` 后的整段 Julia 表达式属于一个参数，不能按空格继续拆开。它先生成 TeX 文件；随后 recipe 可接上使用适当引擎的 latexmk 工具。前处理生成的目录、文件名与下一步输入必须吻合。首次使用时先在终端验证前处理，再组合完整 recipe。

部分语言模式的自动 recipe 匹配还参考名称中的 `jnw`、`rnw` 或相应语言标识，因此不要将这些配方随意命名为无法辨认来源的名称。[S7]

11.3 代码高亮与 shell escape

若模板或旧版 minted 工作流要求调用外部高亮程序，可能需要额外可执行程序和 shell escape 权限。是否需要完整 `-shell-escape` 取决于宏包版本、TeX 发行版与受限命令配置；不能把旧教程中的开关无条件添加到所有论文项目。遇到问题先读当前宏包手册与具体错误。

本教程用不依赖外部高亮进程的 verbatim 方式展示代码，因而重编译教程本身不需要 shell escape。这个选择也便于把“编译配置教学”与“代码高亮环境配置”分开验证。

11.4 非常用设置怎样查

`extraExts` 可把额外后缀纳入插件解析，值通常需包含前导点；`verbatimEnvs` 告知插件哪些环境应按原样文本处理，避免把代码示例中的 LaTeX 命令误判成项目结构。它们改变的是插件解析行为，不会安装新编译器，也不会自动让某个文件格式获得排版能力。[S2]

对于 Tectonic、ConTeXt 或特殊前处理链，继续采用相同的提问顺序：入口文件是什么，程序是什么，输入参数是什么，产生什么中间文件，下一步从哪里读取，以及插件去哪里找最终 PDF。掌握这个方法比背诵一份很长的 settings.json 更容易迁移。

11.5 可计算论文还需要记录什么

把计算代码放入文档，并不自动产生可复现结果。除了排版命令，还需记录数据版本、计算环境、随机种子、生成图表的步骤及运行目录。前处理失败时，旧的 TeX 或图表可能仍然留在磁盘上，应防止后续排版悄悄使用陈旧结果。

一种实用安排是先让独立脚本生成固定名称的结果和图表，检查成功后才调用排版。小项目可以用顺序 recipe；涉及多种依赖、缓存和失败处理时，交给明确的构建脚本或 Makefile 更容易维护。无论选哪种方式，都应规定失败时是否停止、哪些结果可以重建，以及从空输出目录开始能否得到一致的论文内容。

第 12 章：综合实践与独立配置能力

12.1 实践目标

建立含两个章节、一个交叉引用和一条参考文献的小工程。先用终端完成构建，再让 LaTeX Workshop 执行相同流程；随后迁移到 build 输出目录，并验证增量构建和清理。源码包中的 `examples/minimal-paper` 可作为起点。

第一阶段检查程序与模板是否兼容。英文示例使用 pdfLaTeX，不依赖本机中文字体；若改成中文论文，应换为合适模板、字体与引擎。第二阶段用手动四步链观察 `.aux`、`.bbl` 的产生。第三阶段改成 latexmk，比较它在第一次完整构建与无改动重建时的行为。

12.2 实践步骤

1. 打开示例目录，在终端运行 `latexmk -pdf -synctex=1 -outdir=build main.tex`。
2. 检查 `build/main.pdf` 中的章节引用和文献编号，确认没有问号。
3. 打开项目设置，用 Build with recipe 选择 `paper-pdf`，核对插件日志中的输入与输出位置。
4. 仅修改 `chapters/model.tex` 的一句正文，观察构建是否仍以 main.tex 为入口。
5. 再构建一次且不修改输入，观察 latexmk 是否认为无需重建。不要用一次测量推断普遍性能。
6. 执行辅助文件清理，确认原始 `.tex`、`.bib` 和最终 PDF 的状态符合所选清理方法。
7. 若发生异常，按第 8 章记录实际命令、首条错误和路径，而不是一次性粘贴更多配置。

12.3 自测题

题 1。 `name` 与 `command` 能否不同？recipe 的 tools 应引用哪一个？

题 2。 把 outDir 改为 build，但工具仍直接把 PDF 写在主文件旁，预览为什么可能失败？

题 3。 为什么 `xelatex → bibtex → xelatex → xelatex` 比单次 XeLaTeX 更完整，但仍不能保证覆盖所有复杂论文？

题 4。 `onSave`、`onFileChange` 与 latexmk 的 `-pvc` 有什么关系？

题 5。 改了 clean.fileTypes 后清理范围没有变化，先检查哪一项？

题 6。 主文件头有 TEX program 注释，为什么 recipe.default 看起来没有生效？

题 7。 texDirs 中加入一个宏包目录后，为什么编译器仍可能报告文件缺失？

题 8。 配置文件没有语法错误，是否足以证明构建设置正确？

12.4 参考答案

答 1。 可以不同。name 是配置内标识，command 是实际程序；recipe 引用工具的 name。

答 2。 插件查找目录与程序输出目录不一致。应同时核对 outDir 与工具的输出参数，以及有无 jobname 改名。

答 3。 它补充文献处理和读回阶段；复杂目录、索引、术语表或其他依赖可能需要更多步骤。latexmk 能自动处理常见依赖，特殊规则仍需配置。

答 4。 前两者是插件的触发方式，pvc 是 latexmk 自身的长期监视模式。普通插件使用通常由插件监视、latexmk 单次构建后退出。

答 5。 检查 clean.method 是否为 glob；若为 command，应修改相应程序和参数。随后检查项目覆盖和输出目录。

答 6。 程序魔法注释可能走兼容构建路径。移除冲突注释，明确选择 recipe，并以日志核实，不依赖猜测的优先级。

答 7。 插件搜索与 TeX 搜索是两层机制；还需使编译器搜索路径、相对路径或发行版安装位置正确。

答 8。 不能。还需验证程序、工具引用、参数边界、入口文件、路径、文献链条及输出结果。

12.5 形成自己的论文配置

为每篇论文保留一份足够小、经过验证的项目设置，并在 README 中写明引擎、TeX 发行版、文献后端、构建命令和输出位置。遇到新模板，先阅读模板要求，再复用已理解的模块。这样配置文件会成为论文工程的一部分，而不是无法解释的网络粘贴片段。

附录 A：设置参数分类速查

以下条目由读取时的配置 schema 核对，去掉共同前缀 `latex-workshop.` 以便阅读。默认值列记录快照事实；说明列为本书概括。特别长的内置数组以“内置列表”表示，避免把大量机器配置当作阅读正文。完整值请在本机设置页查看。

A.1 工具、配方与选择

参数（省略共同前缀）	官方快照默认值	功能与注意事项
`latex.recipes`	内置列表，9 项	有序编译流程列表；每项引用工具名。
`latex.tools`	内置列表，12 项	程序、参数、环境与工作目录定义。
`latex.recipe.default`	`"first"`	默认 recipe；first、lastUsed 或具体名称。
`latex.build.enableMagicComments`	`true`	启用文件内魔法注释；包括入口提示。
`latex.build.forceRecipeUsage`	`true`	schema 保留项；需核对具体版本行为，不作为通用优先级保证。
`latex.magic.args`	`["-synctex=1","-interaction=nonstopmode","-file-line-error","%DOC%"]`	程序魔法注释路径的默认 TeX 参数。
`latex.magic.bib.args`	`["%DOCFILE%"]`	文献程序魔法注释的默认参数。
`latex.external.build.command`	`""`	外部构建程序；非空时绕过普通 recipes。
`latex.external.build.args`	`[]`	外部构建程序的参数列表。

A.2 文件位置与命名

参数（省略共同前缀）	官方快照默认值	功能与注意事项
`latex.outDir`	`"%DIR%"`	插件查找 PDF、SyncTeX 的位置；需与实际输出一致。
`latex.auxDir`	`"%OUTDIR%"`	插件查找辅助文件的位置；与工具的辅助目录配套。
`latex.jobname`	`""`	匹配工具 jobname 参数；空值沿用通常命名。
`latex.build.fromFolder`	`""`	recipe 工作目录；空值使用主文件目录。
`latex.build.fromWorkspaceFolder`	`false`	旧兼容设置；推荐检查 fromFolder，不假定仍控制当前调用。

接下页

参数（省略共同前缀）	官方快照默认值	功能与注意事项
latex.build.rootfileInStatus	false	状态栏显示正在构建的根文件，便于诊断。
latex.texDirs	[]	插件额外 TeX 搜索目录，使用绝对路径。
latex.bibDirs	[]	文献补全额外搜索目录；不等于编译器搜索路径。

A.3 根文件与插件解析

参数（省略共同前缀）	官方快照默认值	功能与注意事项
latex.rootFile.indicator	"\\documentclass[]{}"	识别根文件的文本标志；常为文档类。
latex.search.rootFiles.include	["**/*.tex","**/*.rnw","**/*.Rnw"]	根文件候选的 glob 范围。
latex.search.rootFiles.exclude	[]	排除的根文件候选范围。
latex.rootFile.useSubFile	true	使用 subfiles 时，非交互操作是否指向子文件。
latex.rootFile.doNotPrompt	false	使用 subfiles 时，交互操作是否跳过询问。
latex.extraExts	[]	纳入插件解析的额外后缀，需含前导点。
latex.verbatimEnvs	["verbatim", "lstlisting", "minted", "filecontents"]	从结构分析中排除的原样文本环境。

A.4 自动构建与文件监视

参数（省略共同前缀）	官方快照默认值	功能与注意事项
latex.autoBuild.run	"onFileChange"	never / onSave / onFileChange。
latex.autoBuild.interval	1000	自动构建最小触发间隔，单位 ms。
latex.autoBuild.onSave.files.ignore	["**/*.sty","**/*.cls"]	仅保存触发机制使用的忽略模式。
latex.watch.files.ignore	内置列表，32 项	文件监视忽略模式；修改时保留必要默认项。
latex.watch.delay	250	确认非二进制文件删除的延迟，单位 ms。
latex.watch.pdf.delay	250	确认 PDF 等二进制变化稳定的延迟，单位 ms。

A.5 日志、警告与通知

参数（省略共同前缀）	官方快照默认值	功能与注意事项
`latex.build.clearLog.everyRecipeStep.enabled`	`true`	每一步前清空编译器输出；false 保留全流程输出。
`latex.option.maxPrintLine.enabled`	`true`	针对适用 MiKTeX 程序添加日志行宽选项。
`message.log.show`	`true`	是否显示插件调试日志。
`message.badbox.show`	`"both"`	badbox 显示范围：both / overfull / underfull / none。
`message.latexlog.exclude`	`[]`	排版日志诊断的正则排除列表。
`message.bibtexlog.exclude`	`[]`	BibTeX 日志诊断的正则排除列表。
`message.biberlog.exclude`	`[]`	Biber 日志诊断的正则排除列表。
`message.dvipdfmxlog.exclude`	`[]`	dvipdfmx 日志诊断的正则排除列表。
`message.information.show`	`false`	普通信息通知开关。
`message.warning.show`	`true`	警告通知开关。
`message.error.show`	`true`	错误通知开关；不消除实际错误。
`message.convertFilenameEncoding`	`true`	日志文件名编码转换；编码问题时结合本机版本检查。

A.6 清理与失败重试

参数（省略共同前缀）	官方快照默认值	功能与注意事项
`latex.clean.method`	`"command"`	command 或 glob；决定哪组参数真正生效。
`latex.clean.command`	`"latexmk"`	command 方法调用的清理程序。
`latex.clean.args`	`["-outdir=%OUTDIR%","-auxdir=%AUXDIR%","-c","%TEX%"]`	清理程序参数；另有调用文件占位符 TEX。
`latex.clean.fileTypes`	内置列表，23 项	glob 方法的匹配列表；不要匹配论文源材料。
`latex.clean.subfolder.enabled`	`false`	是否清理输出/辅助目录的子目录。
`latex.autoClean.run`	`"never"`	自动清理时机：never/onFailed/onSucceeded/onBuilt；依安装版本核对。 快照另含 onSucceeded，Wiki 列表未完整列出。
`latex.autoBuild.cleanAndRetry.enabled`	`true`	失败后清理重试；调试时可暂时关闭。

本附录共核对 49 个设置条目。默认列表并不意味着适合所有论文；本书第 4 章配置是教学建议，不能与此列混为一谈。

附录 B：官方网页学习索引与版本说明

B.1 本书章节与官方入口

本书主题	官方 Compile 页内对应章节	阅读目的
第 1 至 3 章	Building the document / LaTeX recipes / LaTeX tools	理解入口、命令与顺序执行
第 4 章	LaTeX tools；另读 latexmk 手册	区分插件字段与程序参数
第 5 章	Placeholders / outDir / auxDir / jobname	核对文件、目录和命名约定
第 6 章	Multi File projects / The root file / The dependencies	理解主文件识别和依赖
第 7 章	Auto build LaTeX / watch 相关设置	理解触发与监视
第 8 章	Catching errors and warnings	分析日志与诊断显示
第 9 章	Cleaning generated files	区分清理方法与时机
第 10 章	Magic comments / External build command	理解特殊入口与优先级
第 11 章	Building a .jnw file / Building a .rnw file	将前处理组合进 recipe

B.2 已发现的文档与定义差异

Wiki 是持续编辑的说明页，仓库 master 是持续变化的开发状态，二者可能暂时不同。本文不把 master 中出现的选项等同于所有已发布版本的能力。核对时发现：Wiki 的 autoClean.run 列表没有完整列出 schema 中的 `onSucceeded`；部分默认列表在 Wiki 与 schema 中有差异；魔法注释的冲突选择顺序也需结合当前实现理解。

因此，复制配置前先看本机扩展版本和设置补全。对文献后端、目录分离、WSL 与外部脚本，最后必须以实际命令和生成文件验证。本文的示例链路和教程 PDF 已在制作环境运行；没有在真实 VS Code 界面中逐一操作所有设置，R/Julia/Pweave 等扩展示例是结构说明，不宣称已运行验证。

B.3 来源列表

[S1] LaTeX Workshop 官方 Wiki：Compile。主入口及页内各部分；用于组织主题、核对占位符与使用流程。访问日期：2026-10-23。

https://github.com/James-Yu/LaTeX-Workshop/wiki/Compile

[S2] LaTeX Workshop 仓库：package.json。用于核对设置名、类型、默认值与枚举。读取快照声明版本 10.20.0；文件 blob SHA：b694d3e5d2cec64ba4dbcb5088e882f2cc8cad00。

https://github.com/James-Yu/LaTeX-Workshop/blob/master/package.json

[S3] John Collins：latexmk 官方手册，2026-03-09 版。重点为命令行选项、初始化文件、构建依赖与辅助/输出目录。

https://mirrors.ctan.org/support/latexmk/latexmk.txt

[S4] Visual Studio Code：User and workspace settings。用于解释用户/项目作用域、设置编辑与检查方式。

https://code.visualstudio.com/docs/configure/settings

[S5] LaTeX Workshop：src/utils/utils.ts。用于交叉检查工作目录与占位符展开。

https://github.com/James-Yu/LaTeX-Workshop/blob/master/src/utils/utils.ts

[S6] LaTeX Workshop：src/compile/build.ts 与 src/compile/README.md。用于交叉检查自动构建、节流和执行职责。

https://github.com/James-Yu/LaTeX-Workshop/blob/master/src/compile/build.ts

https://github.com/James-Yu/LaTeX-Workshop/blob/master/src/compile/README.md

[S7] LaTeX Workshop：src/compile/recipe.ts。用于核对 recipe 解析和魔法程序调用；读取时文件 blob SHA：3df49cf902071a8a2e591772cf45c4ffaf871223。

https://github.com/James-Yu/LaTeX-Workshop/blob/master/src/compile/recipe.ts

[S8] LaTeX Workshop 官方 Wiki：View。同步与预览属于延伸阅读，本文只解释它们与构建产物的关系。

https://github.com/James-Yu/LaTeX-Workshop/wiki/View

使用与归属。 本书教学叙述、实验和工程布局为重新编写；LaTeX Workshop 的软件及随附文档遵循其 MIT 许可，相关声明随源码包附上。本文不代替论文模板要求及各工具的完整手册。